



THE UNIVERSITY  
*of*  
**WISCONSIN**  
MADISON



ECE/CS/ME 539

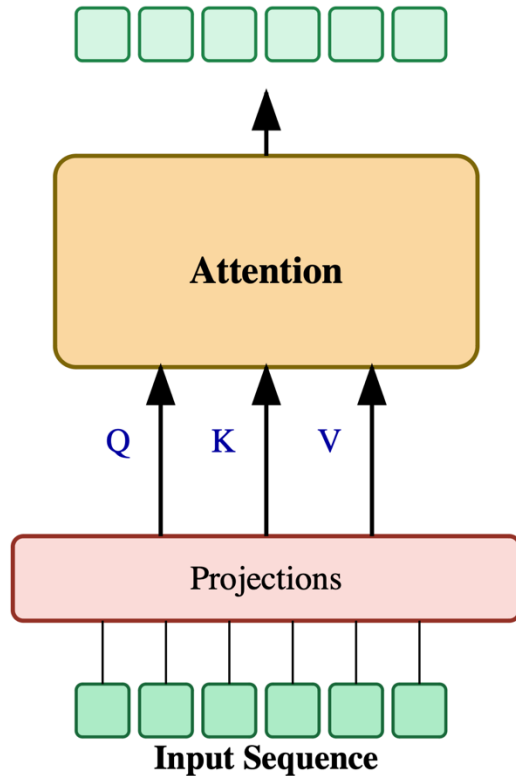
Introduction to Artificial Neural Networks

# Transformer Architecture

## [RECAP] Two types of attention

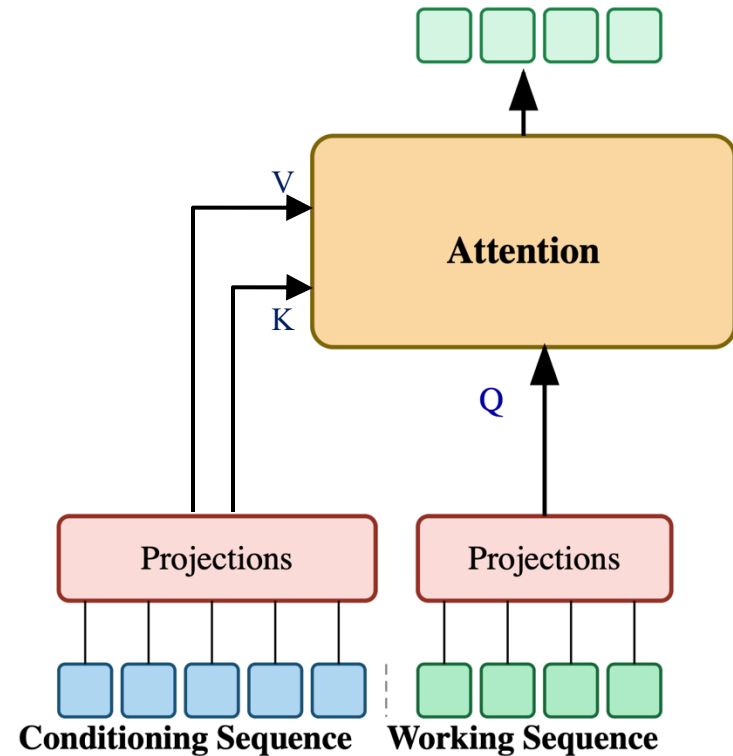
### Self-attention

For processing or transform the working sequence.



### Cross-attention

For incorporating information from another sequence.



# Transformer Encoder

- Stack of alternating **self-attention** and **MLP blocks**.
- Residual connections and LayerNorms

## Self-Attention:

$$\alpha_i = \text{Softmax} \left( \frac{(Qx_i)^T(Kx_1)}{\sqrt{d}}, \dots, \frac{(Qx_i)^T(Kx_n)}{\sqrt{d}} \right)$$

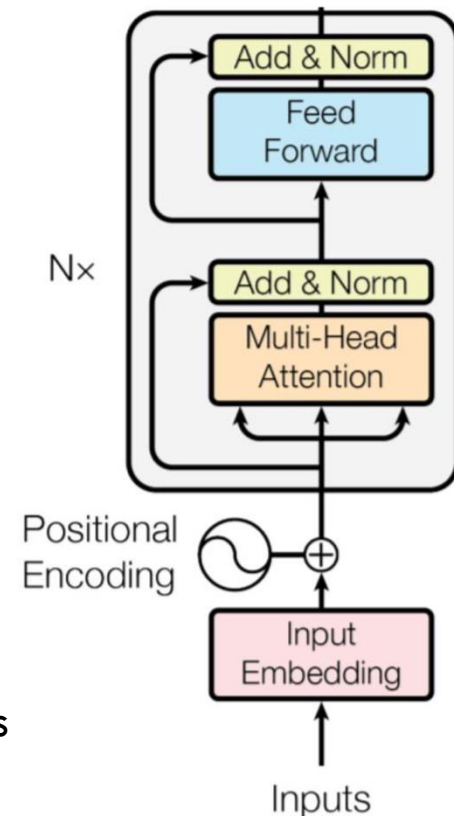
$$o_i = \sum_j \alpha_{ij} Vx_j$$

## Multi-head Self-Attention:

- Uses different QKV projections to attend to different parts of the input.

$$o_i = \sum_{h=1}^H \text{SelfAtt}(x; Q_h, K_h, V_h)$$

**Challenge:** Attention mechanism is invariant to permutations of the input sequence. Position of each token must be embedded in the representation itself.



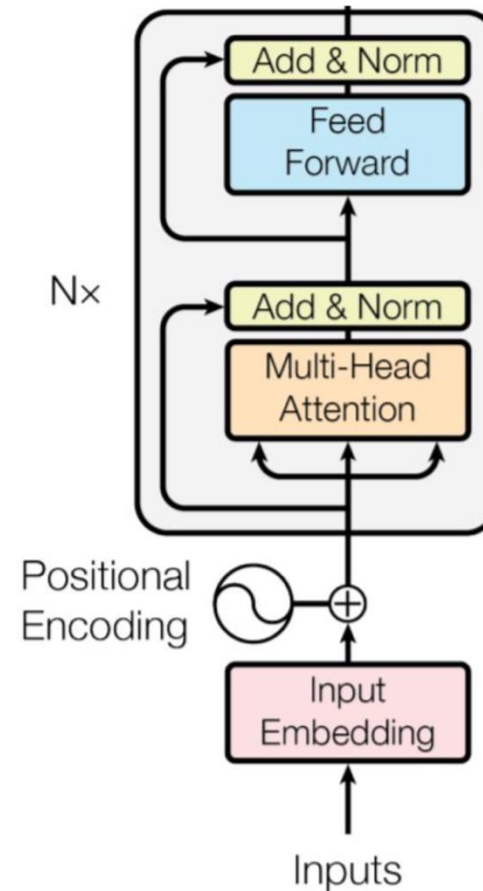
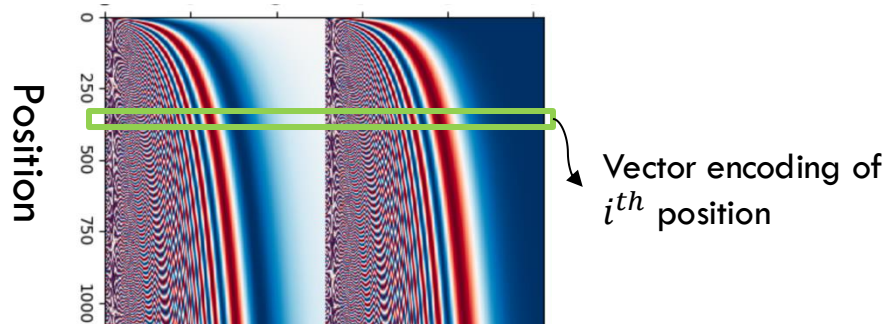
# Transformer Architecture

## Input Embedding:

- Encodes each word in the dictionary into a fixed size vector
- Learned Embeddings, Word2Vec

## Positional Encodings

- Encodes the position of the word within the sentence into a fixed size vector
- SinCos encodings are common

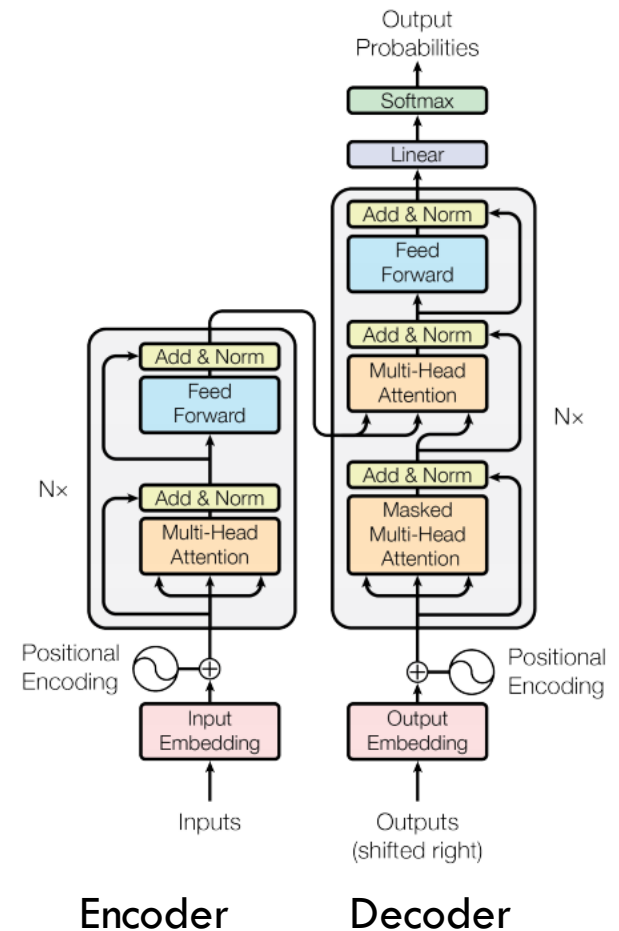
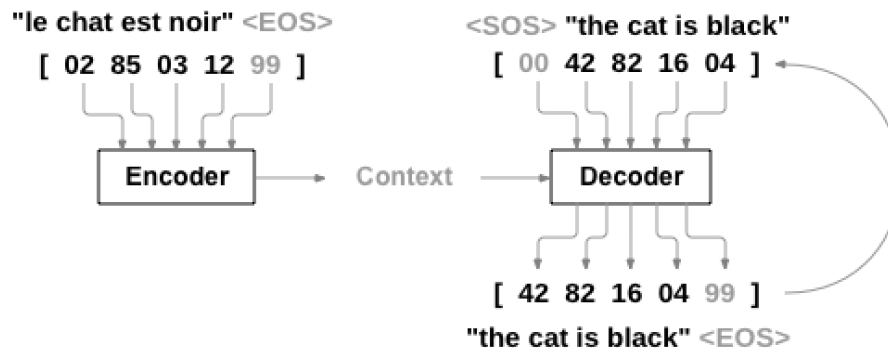


# Transformer Encoder-Decoder

## Useful when output space is different from input space

- Encoder processes input sequence.
- Decoder processes output sequence.

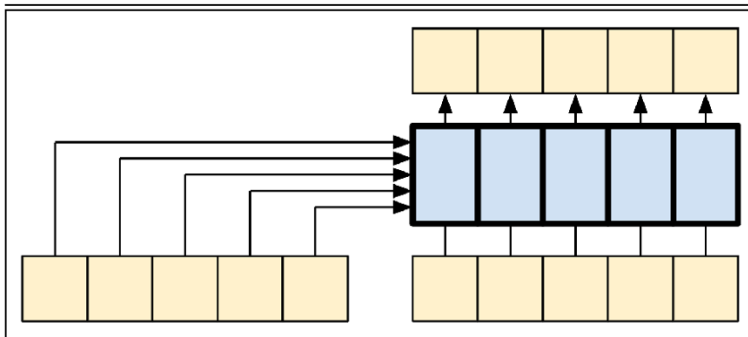
### Example: Translation



# Cross Attention

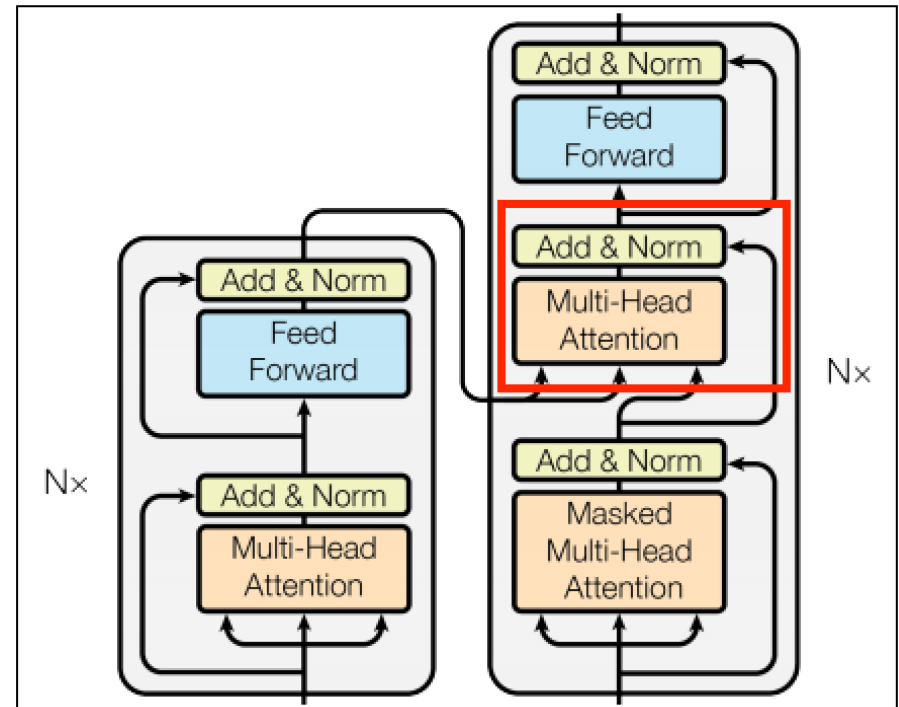
- Query sequence: Encodes the output sequence that is being generated by the decoder
- Key/Value (context) sequence: Encodes the input sequence that provides the context for the generation.

**Each query sees the whole context.**



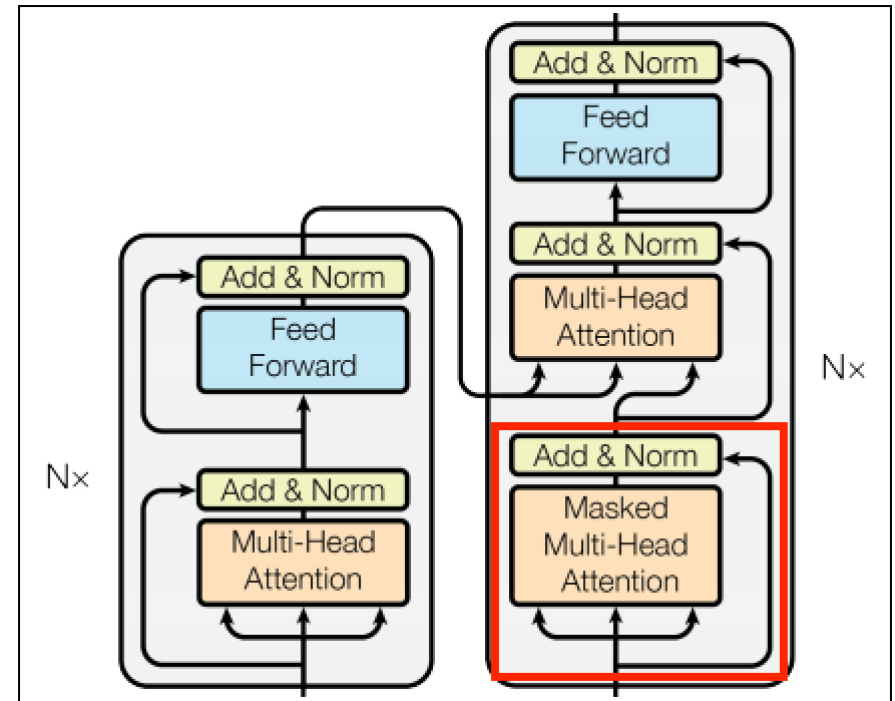
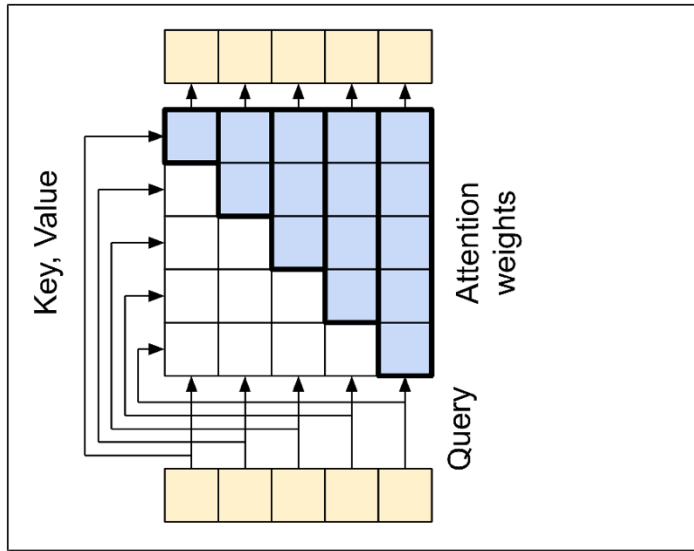
Context sequence  
Output from encoder

Query sequence



# Causal Self-Attention

- Both queries and key/value pairs are derived from same sequence.
- Masked attention ensures that each word can only interact with prior words.
- Good for generation.



# Summary

## Encoder-Decoder Architecture

- Encoder processes input sequence.
  - All tokens can interact with each other (bidirectional context).
- Decoder processes output sequence.
  - Each token can only condition on the past (causal context).
  - Enables language generation.

